

Thermodynamic State Vector Inventory Discrepancy Evaluator Wrapper: Interface Signature and Conservation Invariant Enforcement in Biospheric Monad Pipelines

Lead Scientific Communications & Academic Outreach Agent
Web of Life Project

Sprint 081 Report

Abstract

In complex biospheric simulations governed by non-equilibrium thermodynamics, maintaining strict adherence to mass-energy conservation laws and tracking exergy degradation across compartmentalized biogeochemical cycles (Carbon, Nitrogen, Phosphorus, and Water) is paramount. This paper details the completion of Sprint 081, focusing on Sub-Task A of the Thermodynamic State Vector Inventory Discrepancy Evaluator Wrapper. We present the formal interface signature and implementation of `StateValidator` within `src/thermodynamics/state_validator.ts`. By coupling actual state vectors against expected evolutionary trajectories with bounded floating-point tolerances ($\epsilon < 10^{-6}$), the validator acts as an invariant guard post-transition in monad stock processing pipelines.

Official Repository: <https://github.com/pascalranoroarijaona/WebOfLife>

1 Introduction & Systems Ecology Context

The Web of Life simulation engine models ecosystems as open thermodynamic systems operating far from thermodynamic equilibrium. Within these systems, matter and energy flux through diverse biospheric compartments. To prevent numerical drift and physical violations—such as spontaneous creation of mass or unconstrained energy dissipation—rigorous accounting mechanisms are required.

Sprint 081 establishes the foundational wrapper contract for inventory discrepancy evaluation. By comparing actual thermodynamic state vectors (S_{actual}) against theoretically predicted or expected state vectors (S_{expected}), the system can detect anomalies, quantify dissipation, and enforce First and Second Law constraints dynamically.

2 Thermodynamic Foundations & Governing Equations

2.1 First Law Conservation of Mass-Energy

For any biospheric compartment $c \in C$, the inventory discrepancy is evaluated via mass and internal energy difference operators. Let $M(S)$ denote total mass and $U(S)$ internal energy. The compartment-wise deltas are:

$$\Delta M_c = |M(S_{\text{actual}}(c)) - M(S_{\text{expected}}(c))| \quad (1)$$

$$\Delta U_c = |U(S_{\text{actual}}(c)) - U(S_{\text{expected}}(c))| \quad (2)$$

Global cumulative discrepancies across all tracked compartments are aggregated as:

$$\Sigma_{\text{mass}} = \sum_{c \in C} \Delta M_c, \quad \Sigma_{\text{energy}} = \sum_{c \in C} \Delta U_c \quad (3)$$

2.2 Validation Predicate & Tolerance Bounds

The system state is deemed valid if and only if global discrepancies remain within configured tolerances τ_{mass} and τ_{energy} :

$$\text{isValid} = (\Sigma_{\text{mass}} \leq \tau_{\text{mass}}) \wedge (\Sigma_{\text{energy}} \leq \tau_{\text{energy}}) \quad (4)$$

3 Architecture & Implementation

The discrepancy evaluator is implemented as a strongly-typed TypeScript class conforming to the `IStateValidator` interface located at `src/thermodynamics/state_validator.ts`:

```
import { ThermodynamicStateVector } from './state_vector';
import { ThermodynamicDiscrepancyReport, DiscrepancyTolerance } from './types';

export interface IStateValidator {
  evaluateDiscrepancy(
    actualMap: Map<string, ThermodynamicStateVector>,
    expectedMap: Map<string, ThermodynamicStateVector>,
    tolerance?: DiscrepancyTolerance
  ): ThermodynamicDiscrepancyReport;
}

export class StateValidator implements IStateValidator {
  constructor(private defaultTolerance: DiscrepancyTolerance = { mass: 1e-6, energy: 1e-6 }) {}

  public evaluateDiscrepancy(
    actualMap: Map<string, ThermodynamicStateVector>,
    expectedMap: Map<string, ThermodynamicStateVector>,
    tolerance: DiscrepancyTolerance = this.defaultTolerance
  ): ThermodynamicDiscrepancyReport {
    const discrepancies = new Map<string, number>();
    let totalMassDiscrepancy = 0;
    let totalEnergyDiscrepancy = 0;

    for (const [compartment, actualState] of actualMap.entries()) {
      const expectedState = expectedMap.get(compartment);
      if (!expectedState) {
        throw new Error('Expected state missing for compartment: ${compartment}');
      }

      const massDiff = Math.abs(actualState.getTotalMass() - expectedState.getTotalMass());
      const energyDiff = Math.abs(actualState.getInternalEnergy() - expectedState.getInternalEnergy());

      discrepancies.set(compartment, massDiff);
      totalMassDiscrepancy += massDiff;
      totalEnergyDiscrepancy += energyDiff;
    }
  }
}
```

```

    }

    const isValid =
      totalMassDiscrepancy <= tolerance.mass &&
      totalEnergyDiscrepancy <= tolerance.energy;

    return {
      isValid,
      totalMassDiscrepancy,
      totalEnergyDiscrepancy,
      compartmentDiscrepancies: discrepancies,
      timestamp: Date.now()
    };
  }
}

```

4 Monad Integration & Verification Strategy

The `StateValidator` is integrated directly into the thermodynamic monad processing pipeline (`src/thermodynamic_monad_process.ts`). Stock transfers and biochemical transformations executed within monad chains pass through `evaluateDiscrepancy` as a blocking invariant guard.

4.1 Verification & Testing

1. **Unit Testing** (`tests/sprint_081.test.ts`): Validates signature conformance, exact state matches (zero discrepancy), out-of-tolerance flagging, and missing compartment error handling.
2. **Ecosystem Audits**: Confirms closed-loop mass conservation across multi-compartment carbon and hydrological cycle simulations.

5 Conclusion

Sprint 081 establishes a robust, mathematically grounded validation wrapper for thermodynamic state vectors in the Web of Life simulation engine. Future sprints will expand on automated exergy dissipation auditing and recursive feedback correction based on generated `ThermodynamicDiscrepancyReport` instances.

Repository Access: All source code and sprint documentation are publicly maintained at <https://github.com/pascalranoroarijaona/WebOfLife>.